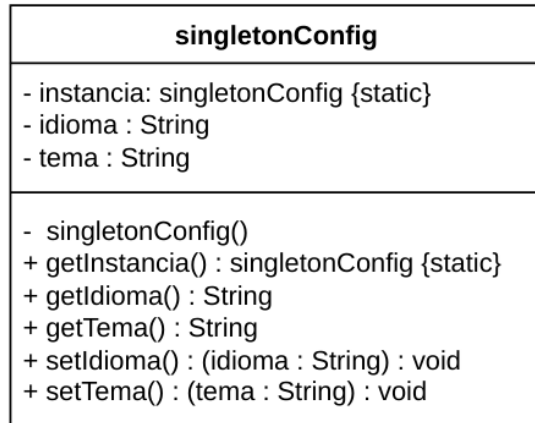


Trabajo Práctico: Patrones de Diseño I

Patrón de Diseño: **Singleton**

1. Diagrama UML de Clases
 - a. Diagrama UML Singleton



- b. Identificar y describir las clases, interfaces, atributos, métodos y relaciones involucradas.
 - **Clase Singleton:** Clase que controla que exista una única instancia
 - **Atributo instancia (Privado y Estático):** Guarda la única instancia de Singleton. Al ser estático, pertenece a la clase y no a los objetos individuales.
 - **Constructor privado (- Singleton()):** Evita que cualquier otra clase externa pueda utilizar el operador de instanciación (new Singleton()).
 - **Método estático (+ getInstance(): Singleton):** Permite obtener la única instancia. Si la variable instancia es nula, crea el objeto; si ya tiene un valor, devuelve la instancia existente.
 - **Interfaz:** No tiene
 - **Relaciones:** No hay otras clases

2. Análisis del Patrón

- a. **Problema:**

En muchas aplicaciones existen recursos o componentes centralizados (como un gestor de configuraciones, un registro de logs o una conexión a una base de datos) donde permitir múltiples instancias genera inconsistencias de datos, uso innecesario de memoria o conflictos de concurrencia. Se necesita garantizar que una clase tenga una y una sola instancia, y proveer un acceso global a ella.

- b. **Solución:**

El patrón restringe la creación de objetos haciendo que el constructor sea privado. Obliga a que la propia clase gestione su única instancia a través de un método estático controlado (getInstance), el cual evalúa si ya fue creada antes de retornarla.

- c. **Consecuencias:**

- Ventajas:
 - Evita que existan múltiples instancias de la clase.
 - Proporciona un único punto de acceso a la instancia.
 - La creación de la instancia puede controlarse desde la propia clase.
- Desventajas / Implicancias:
 - Introduce un acceso global a la instancia.
 - Puede generar un acoplamiento fuerte entre las clases que utilizan el Singleton y esa clase.

- Su uso excesivo puede dificultar el mantenimiento y las pruebas del sistema.

3. Implementación

- Desarrollar un ejemplo de implementación en código para cada patrón de diseño.
- Breve explicación del funcionamiento:
 - La solución implementa el patrón de diseño Singleton, cuyo objetivo es garantizar que exista una única instancia de una determinada clase durante la ejecución de la aplicación.
 - En este caso, la clase singletonConfig representa una configuración general de la aplicación, almacenando el idioma y el tema seleccionado.
 - Para garantizar que solamente exista una instancia, el constructor singletonConfig() se declara como private. De esta manera, otras clases no pueden crear objetos directamente utilizando new singletonConfig().
 - La única forma de obtener la instancia es mediante el método getInstancia(). Este método es static, por lo que puede utilizarse directamente desde la clase sin crear previamente un objeto.
 - Cuando getInstancia() se ejecuta por primera vez, la variable instancia es null, por lo que se crea un nuevo objeto: instancia = new singletonConfig();
 - En las siguientes llamadas, como instancia ya contiene un objeto, no se vuelve a crear otra instancia. Simplemente se devuelve la misma: return instancia;
 - Por ejemplo:


```
singletonConfig config = singletonConfig.getInstancia();
```

 Si posteriormente otra parte del programa realiza:


```
singletonConfig otraConfig = singletonConfig.getInstancia();
```

 config y otraConfig hacen referencia a la misma instancia de singletonConfig.
 -Esto permite mantener una configuración única y compartida, evitando que diferentes partes de la aplicación tengan objetos independientes con valores diferentes.

4.

5. Principios del patrón Singleton

- Una única instancia:** La clase debe garantizar que exista una sola instancia de sí misma.
- Constructor privado:** Evita que otras clases puedan usar el operador de instanciación new.
- Punto de acceso global:** proporciona un método para obtener la instancia, así cualquier parte del programa puede acceder a la única instancia.
- Control centralizado del estado:** mantiene una única configuración, si una parte de la aplicación modifica alguno de los atributos todas las partes que accedan a la misma instancia pueden obtener ese nuevo valor.

6. Principios SOLID cómo se relaciona:

- S — Single Responsibility Principle:** Tiene una responsabilidad concreta: gestionar la configuración global de la aplicación.
- O — Open/Closed Principle:** No demuestra especialmente este principio.
- L — Liskov Substitution Principle:** El principio de Liskov no resulta especialmente relevante en este ejemplo porque el Singleton no utiliza una jerarquía de herencia.
- I — Interface Segregation Principle:** El principio de segregación de interfaces no tiene una aplicación directa en este ejemplo, ya que no se utilizan interfaces.
- D — Dependency Inversion Principle:** es el principio que presenta una relación más problemática con el Singleton. Las clases que utilizan singletonConfig dependen directamente de una clase concreta mediante singletonConfig.getInstancia(). Esto puede aumentar el acoplamiento y dificultar el reemplazo de la implementación.